

교과목 3 : 초거대언어모델(LLM)

단원 3 : LLM

DAY2. LLM 텍스트 생성 제어 파라미터

목 차

1. LLM 주요 파라미터		3
2. AI 에이전트		8
3. AI 앱 vs AI 에이전트		13
4. temperature와 생성 제어		18

1. LLM 주요 파라미터

LLM 의 주요 파라미터는 생성되는 답변의 **창의성**, **후보 단어 선택 범위**, **출력 길이**를 제어하는 설정입니다. 대표적으로 **Temperature**, **Top P**, **Maximum Tokens** 가 있으며, 이들을 적절하게 설정하면 답변의 품질과 비용을 효과적으로 조절할 수 있습니다.

1. Temperature 란?

LLM 은 질문을 받으면 다음에 올 가능성이 높은 단어를 하나씩 선택하여 문장을 생성합니다. 이때 **Temperature** 는 모델이 **얼마나 다양한 단어를 선택할 것인지를 결정하는** 파라미터입니다. Temperature 값이 낮을수록 가장 확률이 높은 단어를 우선적으로 선택하므로 답변이 안정적이고 일관성이 높아집니다. 반대로 Temperature 값이 높을수록 확률이 다소 낮은 단어도 선택할 가능성이 커져 다양한 표현과 창의적인 답변을 생성합니다.

Temperature 설정 예

- **0 ~ 0.2**
 - 항상 비슷한 답변 생성
 - 정확성과 일관성이 매우 높음
 - 분류, 계산, 데이터 추출 등에 적합
- **0.3 ~ 0.5**
 - 자연스러운 답변 생성
 - 대부분의 일반적인 챗봇에서 많이 사용하는 값
- **0.7 ~ 1.0**
 - 다양한 표현 생성
 - 아이디어 생성
 - 광고 문구
 - 소설
 - 창작 작업 등에 적합

Temperature 는 자동차의 **엑셀러레이터**와 비슷합니다.

- 낮으면 안전하게 천천히 운전합니다.
- 높으면 다양한 길을 시도하면서 운전합니다.

즉, Temperature 가 높을수록 모델은 새로운 표현을 많이 시도하게 됩니다.

2. Top P 란?

Top P 는 다음 단어를 선택할 때 사용할 **후보 단어의 범위**를 결정하는 파라미터입니다.
이를 **Nucleus Sampling(누클리어스 샘플링)**이라고도 합니다.

모델은 다음 단어의 확률을 모두 계산합니다.

예를 들어

단어	확률
사과	40%
배	25%
포도	15%
딸기	10%
복숭아	5%
기타	5%

이라고 가정해 보겠습니다.

Top P = 0.5

누적 확률이 50%가 될 때까지 후보를 선택합니다.

후보

- 사과
- 배

만 사용합니다.

Top P = 0.9

누적 확률이 90%가 될 때까지 후보를 사용합니다.

후보

- 사과
- 배
- 포도
- 딸기

등이 모두 선택 대상이 됩니다.

즉,

Top P 값이 작으면 안정적인 단어만 사용하고,

Top P 값이 크면 다양한 단어를 사용할 수 있습니다.

Top P 는 시험 문제를 풀 때 후보 답안을 몇 개까지 볼 것인지를 결정하는 것과 비슷합니다.

- Top P 가 작으면 가장 유력한 답만 봅니다.
- Top P 가 크면 여러 가능성을 함께 고려합니다.

3. Maximum Tokens 란?

Maximum Tokens 는 모델이 생성할 수 있는 최대 출력 길이를 제한하는 설정입니다.

LLM 은 문장을 토큰(Token) 단위로 생성합니다.

Maximum Tokens 를 지정하면 그 수를 넘지 않도록 답변을 생성합니다.

값이 작은 경우

- 짧은 답변 생성
- 비용 절감
- 응답 속도 향상

값이 큰 경우

- 긴 설명 가능
- 상세한 문서 생성 가능
- 비용 증가
- 응답 시간 증가

예를 들어

- 요약문 : 100~300 Tokens
- 일반 설명 : 300~700 Tokens
- 보고서 작성 : 1000 Tokens 이상

등으로 설정할 수 있습니다.

4. 작업 유형별 권장 설정

작업 유형	Temperature	Top P	Maximum Tokens	이유
분류	0~0.2	0.8~1.0	100~300	항상 같은 결과가 중요
정보 추출	0~0.2	0.8~1.0	200~500	정확성이 중요
일반 설명	0.3~0.5	0.8~1.0	500~1000	자연스러운 답변
상담 챗봇	0.3~0.5	0.9	500~1000	유연한 대화
아이디어 생성	0.7~1.0	0.9~1.0	1000 이상	다양성이 중요
소설 작성	0.8~1.0	1.0	2000 이상	창의성이 중요

5. 코드 예제

```
response = client.responses.create(  
    model="gpt-4.1-mini",  
    input="LLM 의 주요 파라미터를 설명해줘.",  
    temperature=0.3,  
    top_p=0.9,  
    max_output_tokens=500  
)  
  
print(response.output_text)
```

위 코드에서는 다음과 같은 설정을 사용합니다.

- **temperature=0.3**
 - 비교적 안정적이고 일관성 있는 답변을 생성합니다.
- **top_p=0.9**
 - 누적 확률 90%에 해당하는 후보 단어들 중에서 다음 단어를 선택하여 자연스러운 다양성을 유지합니다.
- **max_output_tokens=500**
 - 최대 500 개의 토큰까지만 답변을 생성하도록 제한하여 응답 길이와 비용을 관리합니다.

이처럼 세 가지 파라미터를 함께 조정하면 답변의 품질, 창의성, 길이, 처리 비용을 상황에 맞게 효과적으로 제어할 수 있습니다.

핵심 정리

- **Temperature** 는 답변의 창의성과 무작위성을 조절하는 파라미터입니다.
 - 낮을수록 일관성과 정확성이 높아집니다.
 - 높을수록 다양한 표현과 창의적인 답변을 생성합니다.
- **Top P** 는 다음 단어를 선택할 때 사용할 후보 단어의 확률 범위를 결정하는 파라미터입니다.
 - 작을수록 안정적인 단어만 선택합니다.
 - 클수록 다양한 후보 단어를 사용할 수 있습니다.

- **Maximum Tokens** 는 모델이 생성할 수 있는 최대 출력 길이를 제한하는 파라미터입니다.
 - 값이 작으면 짧고 빠른 답변을 생성합니다.
 - 값이 크면 긴 설명과 문서를 생성할 수 있지만 비용과 응답 시간이 증가합니다.
- 일반적으로 **정확성이 중요한 작업**은 Temperature 를 낮게 설정하고, **창의성이 필요한 작업**은 Temperature 를 높게 설정합니다.
- **Top P** 와 **Temperature** 를 함께 조절하면 답변의 다양성과 품질을 세밀하게 제어할 수 있으며, **Maximum Tokens** 를 이용하면 응답 길이와 API 사용 비용까지 효과적으로 관리할 수 있습니다.

2. AI 에이전트

왜 에이전트가 필요한가 — "답은 잘하는데, 일은 못한다"

LLM 은 말은 잘하지만 스스로 행동(도구 사용)은 못 하는 갇힌 두뇌입니다. 여기에 도구·기억·반복 루프를 붙여 실제로 일을 끝내게 만든 것이 바로 에이전트(Agent)이고, 그걸 단계별로 만들어 갑니다.

1. 흔히 겪는 상황

ChatGPT 같은 LLM 에게 "블루투스 이어버드 고르는 법 알려줘"라고 물으면 그럴듯한 답을 술술 내놓습니다. 그런데 우리가 운영하는 쇼핑몰 **승승장구몰**의 상담 현장에서 고객이 진짜로 묻는 건 이런 질문들입니다.

- "제 주문(O000123) 지금 어디까지 왔어요?"
- "이어버드 지금 재고 있어요?"
- "어제 받은 제품 환불하면 언제 입금돼요?"

LLM 은 이 질문에 답할 수 없습니다. 왜냐하면:

- 주문 상태는 우리 회사 주문 DB 에만 있고,
- 재고 수량은 재고 시스템에만 있고,
- 환불 규정은 사내 정책 문서에만 있기 때문입니다.

LLM 은 이 데이터들에 접근할 손발이 없습니다. 학습할 때 본 적도 없습니다. 한마디로 "말은 잘하지만 스스로 아무것도 확인하지 못하는, 갇혀 있는 두뇌" 입니다.

비유하자면, LLM 은 백과사전을 통째로 외운 신입사원입니다. 일반 상식은 막힘없이 답하지만, "우리 창고에 재고 몇 개 있어?"라고 물으면 창고에 갈 수가 없어서 답을 못 합니다. (혹은 더 위험하게, 그럴듯하게 지어냅니다. 이게 4 장에서 배울 '환각'입니다.)

이 "답은 잘하는데 일은 못한다"는 간극(gap)이 바로 이 LLM 교과목 전체가 메우려는 빈칸입니다. LLM 교과목이 끝나면, 위 세 질문에 실제 데이터를 조회해서 답하는 상담 에이전트를 여러분이 직접 만들 수 있게 됩니다.

2. AI 에이전트 등장

Agentic AI(에이전트형 AI) 는 이 두뇌(LLM)에 세 가지를 붙여, **실제로 일을 끝내는 시스템**으로 만든 것입니다.

- **도구(Tool)** — 주문 DB 조회, 재고 확인, 계산, 웹 검색처럼 두뇌가 못 하는 "행동"을 대신해 줍니다.
- **기억(Memory)** — 방금 나눈 대화, 사용자 정보를 잊지 않게 해 줍니다.
- **루프(Loop)** — 한 번에 안 끝나는 일을 "생각 → 행동 → 관찰"을 반복하며 끝까지 처리합니다.

즉, **두뇌 + 손발 + 기억 + 끈기**를 갖춘 것이 에이전트입니다.

구분	LLM 단독	에이전트
구성	두뇌(말만)	두뇌(LLM) + 손발(도구) + 기억(메모리) + 끈기(루프)
결과	"확인은 못 해요"	"확인해서, 끝까지 처리해요"

3. 코드로 확인

LLM 에게 직접 두 가지 질문을 던져 보겠습니다. 하나는 LLM 이 **답할 수 있는** 일반 지식, 다른 하나는 **답할 수 없는** 실시간 사내 정보입니다.

```
# 가장 단순한 LLM 호출 한 번
from google import genai

client = genai.Client(api_key="여러분의_API_키") # Gemini 클라이언트 생성

def ask(question: str) -> str:
    # generate_content = "한 번 묻고 → 한 번 답변기"의 가장 기본 호출
    resp = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=question,
    )
    return resp.text # 모델이 생성한 답변(문자열)

# 질문 1: 일반 지식 → LLM 이 잘 답함
print(ask("블루투스 이어버드를 고를 때 무엇을 봐야 하나요? 3 가지만 짧게."))
```

```
# 질문 2: 실시간 사내 정보 → LLM 혼자서는 불가
print(ask("승승장구물 주문번호 O000123 은 지금 배송 어디까지 왔나요?"))
```

예상 출력

```
[질문 1] 음질, 배터리 지속시간, 연결 안정성을 보세요 ... ← 잘 답함
[질문 2] 죄송하지만 실시간 주문 정보는 확인할 수 없습니다 ... ← 답 못 함
(혹은 그럴듯한 가짜 운송장 번호를 지어냄 = 환각)
```

질문 2 를 제대로 답하려면 LLM 에게 "**주문 조회 도구**"를 쥐여 줘야 합니다. 바로 이 "도구를 붙이는 일"이 에이전트 제작을 위한 프롬프트 엔지니어링이 우리가 직접 하게 될 작업입니다.

위 코드의 `api_key` 는 실제로는 코드에 직접 쓰지 않고 `.env` 파일에서 안전하게 불러옵니다.
모든 실습은 `common.py` 의 `get_genai_client()` 함수가 이 일을 대신해 줍니다.

4. 에이전트 작업 5 단계

"빈 두뇌(LLM) → 진짜로 일하는 상담 에이전트" 까지를 5 개의 큰 단계로 진행됩니다..

1 단계. 두뇌 깨우기 — LLM 을 다루는 법

도구를 붙이기 전에, 먼저 **LLM 자체와 대화하는 법**부터 익힙니다. API 를 호출하고, 원하는 답이 나오도록 프롬프트를 설계하고, 복잡한 문제는 단계적으로 생각하게(추론) 만듭니다.

학습 목적 :

에이전트의 "판단"은 결국 LLM 이 합니다. 두뇌를 제대로 다루지 못하면, 손발(도구)을 달아도 엉뚱하게 씁니다.

2 단계. 손발 달기 — 도구와 행동 (5~12 강)

이 과정의 심장부입니다. LLM 이 외부 함수를 호출하게 만들고(함수 호출), 도구를 여러 개 관리하고, "생각 → 행동 → 관찰"을 스스로 반복하는 **에이전트 루프**를 직접 구현합니다. 이어서 계획 수립, 웹 검색, 코드 실행, 데이터 분석, 여러 도구 통합까지 — "**말만 하던 LLM 이 실제로 일을 하기 시작하는**" 단계입니다.

학습 목적 :

앞서 코드에서 "답 못 했던" 질문 2 를, 도구를 쥐여 주면 답할 수 있게 됩니다. 에이전트의 본질이 여기 있습니다.

3 단계. 지식 주입하기 — RAG(문서 검색)

회사 정책·매뉴얼 같은 **사내 문서**를 LLM 이 근거로 삼아 답하게 만듭니다. 문서를 잘게 쪼개고(칭킹), 의미를 벡터로 바꾸고(임베딩), 벡터 DB 에 저장해, 질문과 관련된 부분만 찾아 **출처와 함께** 답하게 합니다.

"환불 규정이 뭐예요?" 같은 질문은 LLM 이 지어내면 안 됩니다. **회사 문서에 적힌 그대로** 근거를 대고 답해야 신뢰할 수 있는 상담이 됩니다.

4 단계. 기억하게 만들기 — 메모리

LLM 은 기본적으로 **방금 한 말도 잊는(stateless)** 존재입니다. 대화 맥락을 유지하고(단기기억), 세션이 바뀌어도 사용자 정보를 기억하게(장기기억) 만듭니다.

"아까 말한 그 주문이요"라고 했을 때 "어떤 주문이요?"라고 되묻는 봇은 쓸모가 없습니다. 진짜 상담원처럼 **맥락을 이어가야** 합니다.

5 단계. 팀으로 키우고 제품으로 내보내기 — 협업·운영

마지막은 **규모와 완성도**입니다. 흐름을 그래프로 선언하고(워크플로우), 리서치·코딩·비즈니스 같은 전문 에이전트를 만들고, 여러 에이전트가 **팀으로 협업**하게 합니다. 그리고 실제 서비스를 위한 설계·배포·운영을 거쳐, **모든 조각을 하나로 합친 완성형 상담 에이전트**를 만듭니다.

혼자 다 하는 만능 에이전트는 어느 순간 한계에 부딪칩니다. **역할을 나눠 협업**시키고, **내 PC 밖에서도 안정적으로 돌아가게** 만들어야 비로소 "제품"이 됩니다.

단계	작업 내용	끝나면 할 수 있는 것
① 두뇌 깨우기	LLM 호출·프롬프트·추론	LLM 을 의도대로 부린다
② 손발 달기	도구·함수 호출·루프	실시간 정보를 조회해 답한다
③ 지식 주입	RAG(문서 검색)	사내 문서를 근거로 답한다
④ 기억하기	단기·장기 메모리	맥락을 이어 상담한다
⑤ 협업·운영	멀티 에이전트·배포	팀으로 일하는 제품을 완성한다

5. 여러분이 얻는 것

- **개념:** "AI 에이전트가 대체 뭔데?"라는 질문에, 구조를 설명할 수 있습니다.
- **기술:** 도구 호출·RAG·메모리·멀티 에이전트를 **직접 코드로** 조립할 수 있습니다.
- **판단력:** "이 문제엔 RAG 가 필요할까? 멀티 에이전트가 과한가?"를 **트레이드오프로 따져** 결정할 수 있습니다.
- **결과물:** 작동하는 **상담 에이전트**가 결과물로 남습니다.

핵심 정리

- LLM 은 일반 지식엔 강하지만, 실시간·사내 정보엔 무력합니다. ("답은 잘하는데 일은 못한다")
- 그 한계를 **도구·기억·루프**로 메운 것이 **에이전트**입니다.
- 두뇌 깨우기 → 손발 달기 → 지식 주입 → 기억하기 → **협업·운영**의 5 단계로, 빈 LLM 을 진짜로 일하는 **상담 에이전트**로 키워 갑니다.

3. AI 앱 vs AI 에이전트

AI 앱 vs AI 에이전트 — "누가 다음 행동을 결정하는가"

AI 앱은 사람이 매번 다음 할 일을 정해 주는 1 회성 도구이고, AI 에이전트는 LLM 이 스스로 "다음에 뭘 할지" 정하며 목표를 끝낼 때까지 반복하는 시스템입니다.

1. 두 가지를 나란히 놓고 보기

구분	AI 애플리케이션 (AI App)	AI 에이전트 (Agent)
흐름	입력 → LLM → 출력 (1 회로 끝)	목표 → 계획·도구사용·관찰을 반복 → 완료
제어 주체	사람이 매번 지시	AI 가 다음 행동을 스스로 결정
외부 연결	거의 없음	도구(검색·DB·코드실행·API) 사용
예시	번역기, 요약기, 단순 챗봇	주문 조회 상담봇, 리서치봇, 코딩봇

핵심 차이는 딱 하나, "누가 다음에 무엇을 할지 결정하는가" 입니다.

- AI 앱 → 사람이 정합니다. ("이거 번역해줘" → 번역 끝. 다음은 또 사람이 시켜야 함)
- AI 에이전트 → LLM 이 정합니다. ("환불해줘" → LLM 이 스스로 '주문 조회 → 정책 확인 → 안내' 순서를 짜서 실행)

비유: AI 앱은 시키는 메뉴만 만드는 자판기입니다. AI 에이전트는 "오늘 저녁 알아서 차려줘"라고 하면 냉장고를 열고(도구), 어제 뭘 먹었는지 떠올리고(기억), 장을 봐 와서(반복), 상을 차리는 요리사입니다.

2. 코드로 보는 차이

(가) AI 앱 — 한 번 호출하고 끝

```
from google import genai

client = genai.Client(api_key="...")

# 사람이 질문 → LLM 이 답 → 끝. 다음 행동은 사람이 또 정해야 함.
resp = client.models.generate_content(
    model="gemini-2.5-flash",
```

```
contents="이 문장을 영어로 번역해줘: 안녕하세요",
)

print(resp.text)  # "Hello" → 여기서 종료
```

흐름이 **일직선**입니다. 입력 한 번, 출력 한 번. LLM 은 "그 다음에 뭘 할지" 고민하지 않습니다.

(나) AI 에이전트 — 스스로 판단하고 반복 (개념 미리보기)

아래는 agent loop 구조를 **개념만** 보여 주는 의사코드(pseudo-code)입니다.

```
# 개념 미리보기 — 에이전트의 뼈대

목표 = "주문 O000123 의 배송 상태를 알려줘"
대화기록 = [목표]

while True:                                # 목표가 끝날 때까지 반복
    결정 = LLM 에게_묻기(대화기록)         # ① LLM 이 "다음에 뭘 할지" 스스로 결정

    if 결정.종류 == "도구사용":
        결과 = 도구실행(결정.도구, 결정.입력) # ② 도구로 실제 행동 (DB 조회 등)
        대화기록.append(결과)                # ③ 관찰 결과를 기억에 추가 → 다시 ①로

    else:
        print(결정.최종답변)                # ④ 더 할 일이 없으면 답하고 종료
        break
```

여기서 **while** 루프와 **LLM 에게_묻기**가 **매번 다음 행동을 고른다는** 점이 AI 앱과의 결정적 차이입니다.

3. 에이전트가 항상 정답은 아니다

단순 번역·요약처럼 한 번에 끝나는 일은 AI 앱이 더 빠르고 싸고 안전합니다. 에이전트는 "여러 단계를 거쳐야 하고, 도구가 필요한 일"에 어울립니다. 무엇을 만들지는 **문제의 성격**이 정합니다. 이 판단 기준은 '프로젝트 설계'에서 자세히 다룹니다.

핵심 정리

- AI 앱 = 사람이 운전, AI 에이전트 = AI가 운전.
- 에이전트의 특징은 반복 루프 + 스스로 다음 행동 결정.
- 모든 일에 에이전트가 정답은 아니며, 문제에 맞게 고릅니다.

4. 에이전트의 4 요소 — LLM · 도구 · 기억 · 루프

모든 에이전트는 두뇌(LLM)·손발(도구)·기억(메모리)·끈기(루프) 네 부품의 조합입니다.

1. 전체 그림

에이전트의 동작 흐름 (ReAct 루프)

1. 사용자 질문이 들어온다.
2. LLM 두뇌 — "지금 무엇을 해야 하나?"를 판단해 쓸 도구를 결정한다.
3. 도구(Tools) — 검색·DB·계산·코드 실행 등 실제 행동을 수행한다.
4. 실행 결과를 관찰해 기억에 쌓고, 다시 2 번(LLM 판단)으로 돌아간다.
5. 2~4 를 목표가 끝날 때까지 반복한다(루프).
6. 완료되면 최종 답변을 내놓는다.

2. 네 가지 부품

① LLM (두뇌) — 판단하는 자

자연어를 이해하고 "지금 무엇을 해야 하는가"를 결정합니다. 단, 두뇌일 뿐이라 직접 DB 를 뒤지거나 계산을 정확히 하지는 못합니다.

```
# LLM 은 "판단"을 한다 — 어떤 도구를 쓸지 고르는 일
resp = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="주문 O000123 배송 상태 알려줘",
```

```
)  
# → 모델은 "주문 조회 도구를 써야겠다"는 판단을 내릴 수 있다
```

② Tools (도구) — 행동하는 손발

LLM 이 못 하는 능력을 보완합니다. 실시간 정보 조회, 정확한 계산, 코드 실행, 웹 검색 등.
에이전트의 능력 = 어떤 도구를 쥐여 주느냐로 결정됩니다.

```
# 도구는 그냥 '평범한 파이썬 함수'입니다 (LLM 에 연결)  
def get_order_status(order_id: str) -> str:  
    """주문번호로 배송 상태를 조회한다."""  
    # 실제로는 DB/CSV 를 조회 — 여기선 개념만  
    return "배송중 (대구 허브 도착)"
```

③ Memory (기억) — 맥락을 잇는 자

LLM 은 기본적으로 **금붕어**입니다. 방금 한 말도 다음 호출에서 잊습니다. 그래서 대화 기록과 사용자 정보를 따로 저장해 매번 같이 넘겨줘야 합니다.

```
# 기억이 없으면 — 매번 처음 보는 사람 취급  
대화기록 = []  
대화기록.append("내 이름은 임도현이야")  
# 다음 호출 때 이 기록을 통째로 같이 넘겨야 LLM 이 이름을 "기억"한다
```

④ Loop (루프) — 끝까지 해내는 끈기

한 번에 안 끝나는 일을 **생각 → 행동 → 관찰**을 반복하며 처리합니다. "재고 확인 → 부족하면 재주문 기준 조회 → 판단" 같은 다단계 작업이 가능해집니다.

```
# 루프가 핵심 — 목표가 끝날 때까지 반복  
while not 완료:  
    행동 = LLM_판단()  
    결과 = 도구실행(행동)  
    대화기록.append(결과) # 관찰을 기억에 쌓고 다시 위로
```


부품	배우는 것
LLM(두뇌)	API 호출, 프롬프트, 추론
Tools(손발)	함수 호출, 도구 설계
Loop(끈기)	ReAct 루프, 계획
Memory(기억)	단기/장기 기억
(지식 보강)	RAG — 문서를 도구처럼

4. temperature 와 생성 제어

1. 왜 LLM API 를 알아야 하는가

API 를 정확히 다루면 **비용을 아끼고(토큰), 말투를 제어하고(시스템 지시), 결과의 일관성을 조절(temperature)** 할 수 있습니다. 이것이 앞으로 만들 모든 에이전트의 기본기입니다.

처음 LLM API 사용 코드는 이것이 전부였습니다.

```
resp = client.models.generate_content(model="gemini-2.5-flash", contents="안녕")
print(resp.text)
```

그런데 이걸 **회사 서비스**로 만들려면 다음 질문이 쏟아집니다.

- 상담봇 말투를 항상 **친절한 존댓말로** 고정하려면? → **시스템 지시**
- 사용자가 10 만 명이면 **요금이 얼마나** 나올까? → **토큰**
- 똑같이 물어도 답이 매번 달라지는데, **일관되게** 하려면? → **temperature**
- GPT 로 바꾸면 코드를 다 새로 짜야 하나? → **요청-응답 구조의 공통점**

이 네 가지를 모르면, 동작은 하지만 **통제할 수 없는** 챗봇이 됩니다. 2 강은 바로 이 통제력을 손에 넣는 강의입니다.

2. LLM 을 '함수'로 보기

파이썬 개발자에게 가장 친숙한 비유는 **함수**입니다. LLM API 호출은 결국 이런 함수와 같습니다.

```
def llm(요청) -> 응답:
...
```

- **요청(Request)** 에는 세 가지를 담습니다: **어떤 모델(model), 무슨 말(contents), 어떤 설정(config** — 역할·온도 등).
- **응답(Response)** 에는 두 가지가 옵니다: **생성된 텍스트(text), 그리고 사용량 정보(usage_metadata** — 토큰 수).

```
# [요청] model(어떤 모델) + contents(무슨 말) + config(어떤 설정)
resp = client.models.generate_content(
    model="gemini-2.5-flash",          # 모델
    contents="환불 얼마나 걸려요?",  # 입력
    config=...,                       # 설정(역할·온도 등)
```

```
)  
# [응답] resp.text(생성된 답) + resp.usage_metadata(토큰 사용량)
```

이 "요청 3 종 세트 → 응답 2 종 세트" 구조는 **Gemini** 든 **OpenAI** 든 거의 똑같습니다. 그래서 하나만 제대로 익히면 다른 모델로 갈아타기도 쉽습니다.

3. LLM API 4 가지 통제력

통제력	도구	효과
말투·역할 고정	시스템 지시	항상 일관된 페르소나
비용 예측·절감	토큰(usage)	요금을 숫자로 파악
일관성 ↔ 다양성	temperature	분류는 일관, 창작은 다양
모델 갈아타기	공통 요청-응답 구조	Gemini ↔ OpenAI

4. 요청-응답 구조

LLM 호출은 **요청(Request)**을 보내면 **응답(Response)**이 오는 단순한 왕복입니다. 요청에는 model·contents·config 세 가지를, 응답에서는 text·usage_metadata 두 가지를 꺼냅니다. 이 구조를 외우면 나머지는 응용입니다.

우리가 채우는 **요청(Request)**과 모델이 채워 주는 **응답(Response)**은 각각 이렇게 구성됩니다.

요청(Request) — 우리가 채움	응답(Response) — 모델이 채움
model: "gemini-2.5-flash"	text: "평균 3~5 일 소요됩니다..."
contents: "환불 얼마나 걸려요?"	usage_metadata.prompt_token_count = 12
config: system_instruction, temperature	usage_metadata.candidates_token_count = 30
	usage_metadata.total_token_count = 42

우리가 채우는 건 **요청** 세 가지, 모델이 채워 주는 건 **응답** 두 가지(텍스트·사용량)입니다.

5. 요청 3 종 세트

① model — 어떤 두뇌를 쓸까

사용할 모델 이름입니다. common.py 에 정해 둔 GEMINI_MODEL(기본값 "gemini-2.5-flash")을 씁니다. flash 는 **빠르고 저렴한** 모델이라 실습과 대부분의 실무에 적합합니다.

```
from common import GEMINI_MODEL
print(GEMINI_MODEL)  # gemini-2.5-flash
```

② contents — 모델에게 보낼 말

사용자의 질문이나 지시입니다. 가장 단순하게는 그냥 문자열을 넣습니다.

```
contents = "환불 얼마나 걸려요?"
```

③ config — 생성 방식 설정

답변의 **역할·말투·무작위성**을 조절하는 설정 묶음입니다. types.GenerateContentConfig 로 만듭니다.

```
from google.genai import types

config = types.GenerateContentConfig(
    system_instruction="너는 승승장구몰의 친절한 CS 상담원이다.", # 역할
    temperature=0.3, # 무작위성
)
```

세 가지를 합친 한 번의 호출

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

resp = client.models.generate_content(
    model=GEMINI_MODEL, # ① 모델
    contents="환불 얼마나 걸려요?", # ② 입력
```

```

config=types.GenerateContentConfig(      # ③ 설정
    system_instruction="너는 친절한 CS 상담원이다.",
    temperature=0.3,
),
)

```

6. 응답 2 종 세트

① resp.text — 생성된 답변

모델이 만든 텍스트입니다. 우리가 실제로 사용자에게 보여 줄 내용입니다.

```
print(resp.text)    # "환불은 영업일 기준 평균 3~5 일 정도 소요됩니다 ..."
```

② resp.usage_metadata — 토큰 사용량

이번 호출에 **토큰이 몇 개** 쓰였는지 알려 줍니다. **토큰은 곧 요금**이므로 비용을 가늠하는 핵심 정보입니다.

```

u = resp.usage_metadata

print(u.prompt_token_count)      # 입력(질문+시스템지시)에 쓴 토큰
print(u.candidates_token_count)  # 출력(답변)에 쓴 토큰
print(u.total_token_count)       # 합계

```

직접 확인해 보기

```

resp = client.models.generate_content(
    model=GEMINI_MODEL,
    contents="승승장구물을 한 문장으로 홍보해줘.",
    config=types.GenerateContentConfig(temperature=0.7),
)

print("답변:", resp.text)
print("토큰:", resp.usage_metadata)

```

예상 출력

답변: 일상에 필요한 모든 것을 합리적인 가격에! 승승장구몰에서 만나보세요.

토큰: prompt_token_count=11 candidates_token_count=24 total_token_count=35 ...

7. 시스템 지시로 역할 부여

시스템 지시(system instruction) 는 모델에게 미리 주는 "너는 누구이고, 어떻게 답해야 한다"는 규칙입니다. 같은 질문이라도 시스템 지시만 바꾸면 **말투·성격·답변 형식이 통째로** 바뀝니다.

대화에는 두 종류의 말이 있습니다.

- **사용자 지시(user)**: 그때그때 던지는 질문 — "환불 얼마나 걸려요?"
- **시스템 지시(system)**: 대화 전체에 깔리는 **기본 규칙** — "너는 친절한 상담원이고, 항상 존댓말로 답해."

비유하자면, 시스템 지시는 **배우에게 주는 배역 설명서**입니다. "당신은 무뚝뚝한 형사입니다"라고 정해 두면, 어떤 대사를 쳐도 그 캐릭터를 유지하죠. 사용자 질문이 바뀌어도 역할은 그대로 유지됩니다.

1. 코드로 역할 입히기

config 의 system_instruction 에 역할을 적습니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

def chat(system: str, user: str, temperature: float = 0.3) -> str:
    """시스템 지시(system)와 사용자 입력(user)을 받아 답변 텍스트를 돌려준다."""
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=user,                                # 사용자 질문
        config=types.GenerateContentConfig(
            system_instruction=system,                  # ← 역할/규칙
```

```

        temperature=temperature,
    ),
)
return resp.text

```

이제 **같은 질문에 다른 역할**을 입혀 봅시다.

```

q = "환불 얼마나 걸려요?"

print("[친절한 상담원]")
print(chat("너는 승승장구몰의 친절한 CS 상담원이다. 존댓말로 간결히 답하라.", q))

print("\n[무뚝뚝한 엔지니어]")
print(chat("너는 무뚝뚝한 엔지니어다. 핵심만 한 줄로 답하라.", q))

```

예상 출력

```

[친절한 상담원]
안녕하세요 고객님! 환불은 영업일 기준 평균 3~5 일 정도 소요됩니다. 도움이 더 필요하시면
말씀해 주세요 :)

[무뚝뚝한 엔지니어]
환불 3~5 영업일.

```

질문은 똑같은데 **말투·길이·태도**가 완전히 달라졌습니다. 이게 시스템 지시의 힘입니다.

2. 좋은 시스템 지시 작성법

시스템 지시는 보통 세 가지를 담으면 좋습니다.

요소	예시
역할(누구)	"너는 승승장구몰의 CS 상담원이다."
태도(어떻게)	"항상 존댓말로 공감하며 간결히 답하라."
안전장치(모르면)	"확실하지 않으면 '확인 후 안내드리겠습니다'라고 답하라."

ROLE = (

"너는 승승장구몰의 친절한 CS 상담원이다. " # 역할

"고객 문의에 존댓말로 공감하며 간결하게 답하라. " # 태도

"확실하지 않은 정보는 '확인 후 안내드리겠습니다'라고 답하라." # 안전장치

)

세 번째 "안전장치"가 특히 중요합니다. **환각을 줄이는 첫 번째 방어선**이기 때문입니다.

"모르면 모른다고 말하라"는 지시 한 줄이 엉뚱한 답을 크게 줄입니다.

3. 시스템 지시 vs 사용자 지시, 무엇이 우선인가

시스템 지시는 사용자 지시보다 **우선**하도록 설계돼 있습니다. 그래서 "너는 존댓말만 쓴다"는 시스템 지시가 있으면, 사용자가 "반말로 해"라고 해도 어느 정도 존댓말을 유지하려 합니다. (단, 100% 보장은 아닙니다. 악의적으로 규칙을 깨려는 입력을 **프롬프트 인젝션**이라 합니다.)

8. 토큰과 비용 감각

LLM 은 글자가 아니라 **토큰(token)** 단위로 글을 읽고 씁니다. 그리고 **요금은 토큰 수로 매겨집니다**. 토큰을 이해하면 "이 서비스 월 얼마 나올까?"를 숫자로 예측하고, 비용을 줄일 수 있습니다.

1. 토큰이란? — 글을 자르는 단위

LLM 은 문장을 통째로 보지 않고, **토큰**이라는 작은 조각으로 잘라서 처리합니다. 토큰은 대략 "단어 또는 단어의 일부"입니다.

- 영어: "earbuds" → 보통 1~2 개 토큰
- 한국어: "이어버드" → 보통 더 잘게 쪼개져 여러 토큰

중요한 사실 하나: **같은 뜻이라도 한국어가 영어보다 토큰을 더 많이 먹습니다**. 한글은 영어보다 잘게 쪼개지는 경향이 있기 때문입니다. 이걸 비용에 직접 영향을 줍니다.

비유: 토큰은 **택시 미터기**입니다. 거리(글자 수)가 아니라 미터기가 찍는 횟수(토큰 수)로 요금이 매겨집니다. 그리고 한국어는 미터기가 더 자주 찍히는 길이라고 보면 됩니다.

2. 입력 토큰 vs 출력 토큰

요금은 두 가지로 나뉩니다.

- **입력(prompt) 토큰**: 우리가 보낸 것 — 시스템 지시 + 질문 + (나중엔) 대화 기록·검색 결과
- **출력(candidates) 토큰**: 모델이 생성한 답변

보통 **출력 토큰이 입력보다 비쌉니다**. 그래서 "답을 짧게" 시키는 것도 비용 절감의 한 방법입니다.

```
u = resp.usage_metadata

print(u.prompt_token_count)      # 입력 토큰 (내가 보낸 것)
print(u.candidates_token_count)  # 출력 토큰 (모델이 만든 것)
print(u.total_token_count)       # 합계
```

3. 직접 토큰을 세어 보자 (한국어 vs 영어)

같은 의미의 문장을 한국어와 영어로 각각 보내, 토큰 수를 비교해 보겠습니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

def count_tokens(text: str):
    """문장 하나를 보내 입력/출력/합계 토큰을 돌려준다."""
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=text,
        config=types.GenerateContentConfig(temperature=0),
    )
    u = resp.usage_metadata
    return u.prompt_token_count, u.candidates_token_count, u.total_token_count

pairs = {
```

```

"한국어": "승승장구몰의 무선 블루투스 이어버드를 한 문장으로 친절하게 홍보해줘.",
"영어": "Write one friendly sentence promoting SeungSeung Mall's wireless earbuds.",
}

for lang, text in pairs.items():
    in_tok, out_tok, total = count_tokens(text)
    print(f"[{lang}] 입력={in_tok} / 출력={out_tok} / 합계={total}")

```

예상 출력

```

[한국어] 입력=21 / 출력=18 / 합계=39
[영어]   입력=15 / 출력=16 / 합계=31

```

같은 뜻인데도 **한국어 입력 토큰이 더 많죠?** 이게 "한국어가 더 비싸다"의 실체입니다.

4. 비용을 숫자로 추정하기

토큰 수에 **단가**를 곱하면 비용이 나옵니다. 단가는 보통 "토큰 100 만 개당 몇 달러" 형식입니다.

```

# [경고] 아래는 '예시 단가'입니다. 실제 요금은 모델·시점별 공식 단가표를 확인하세요.
PRICE_IN_PER_1M = 0.075  # 입력 100 만 토큰당 $0.075 (예시)
PRICE_OUT_PER_1M = 0.30  # 출력 100 만 토큰당 $0.30 (예시)

def estimate_cost(in_tok: int, out_tok: int) -> float:
    """1 회 호출 비용(USD)을 예시 단가로 계산한다."""
    return in_tok / 1_000_000 * PRICE_IN_PER_1M + out_tok / 1_000_000 * PRICE_OUT_PER_1M

# 1 회는 푼돈처럼 보이지만, 월 호출량을 곱하면 진짜 비용이 보인다
per_call = estimate_cost(21, 18)  # 한국어 기준
monthly  = per_call * 10_000      # 월 1 만 건

print(f"1 회 ≈ ${per_call:.6f}")
print(f"월 10,000 건 ≈ ${monthly:.2f} (약 {monthly * 1385:,.0f}원, 환율 1385 가정)")

```

예상 출력

1 회 $\approx$ \$0.000007

월 10,000 건 $\approx$ \$0.07 (약 97 원, 환율 1385 가정)

flash 모델은 이처럼 매우 저렴합니다. 하지만 **답변이 길어지거나(출력 토큰 $\uparrow$), 대화 기록·검색 결과가 쌓이면(입력 토큰 $\uparrow$)** 비용은 빠르게 커집니다. 이 감각을 미리 가지고 있어야 나중에 RAG 나 긴 대화에서 비용 폭탄을 피합니다.

5. 비용을 줄이는 실전 팁

- **답을 짧게 시킨다:** "3 줄 이내로 답하라" $\rightarrow$ 출력 토큰 절감(가장 비싼 부분).
- **시스템 지시를 간결하게:** 매 호출마다 입력 토큰으로 따라붙는다.
- **대화 기록을 다듬는다:** 오래된 메시지는 잘라낸다.
- **싼 모델을 기본값으로:** 어려운 작업만 큰 모델로(가성비 라우팅).

9. temperature 와 생성 제어

temperature(온도)는 답변의 무작위성을 조절하는 손잡이입니다. **0 에 가까우면 일관되고 안정적, 높으면 다양하고 창의적**입니다. 분류·계산 같은 일은 0, 홍보 문구·아이디어 같은 일은 0.7 처럼 씁니다.

1. temperature 란

LLM 은 "다음에 올 가장 그럴듯한 단어"를 고른다고 했습니다. 이때 **얼마나 모험적으로 고를지**를 정하는 게 temperature 입니다.

- **낮은 temperature(0~0.2):** 항상 "가장 확률 높은 단어"만 고름 $\rightarrow$ 매번 거의 같은 답 (일관·안정)
- **높은 temperature(0.7~1.0):** 확률이 좀 낮은 단어도 과감히 시도 $\rightarrow$ 다양하고 창의적인 답

비유: temperature 는 주사위의 크기입니다. 0 이면 주사위를 안 던지고 늘 1 번 답만 말합니다. 높을수록 큰 주사위를 굴러 색다른 답이 나올 여지를 줍니다.

2. 작업마다 적절한 온도가 다르다

작업 유형	권장 temperature	이유
분류, 추출, 계산	0	정답이 정해져 있어 일관성 이 중요
일반 상담, 설명	0.3	자연스럽되 너무 튀지 않게
홍보 문구, 아이디어	0.7 ~ 1.0	다양성 이 곧 가치

에이전트에서 **도구를 고르거나 데이터를 분류**하는 일은 거의 항상 temperature=0 을 씁니다.
"이 질문은 환불 카테고리"라는 판단이 호출할 때마다 바뀌면 곤란하니까요.

3. 코드로 온도의 효과 보기

같은 질문을 **온도만 바꿔** 여러 번 호출해, 답이 얼마나 다양해지는지 봅니다.

```
from common import get_genai_client, GEMINI_MODEL
from google.genai import types

client = get_genai_client()

def promote(temperature: float) -> str:
    resp = client.models.generate_content(
        model=GEMINI_MODEL,
        contents="승승장구물을 한 문장으로 홍보해줘.",
        config=types.GenerateContentConfig(temperature=temperature),
    )
    return resp.text

# 온도 0: 여러 번 불러도 거의 같은 답
print("[temperature=0]")
for _ in range(3):
    print("-", promote(0.0))

# 온도 1.0: 부를 때마다 색다른 답
print("\n[temperature=1.0]")
for _ in range(3):
    print("-", promote(1.0))
```

예상 출력

[temperature=0]

- 일상에 필요한 모든 것을 합리적인 가격에, 승승장구물!
- 일상에 필요한 모든 것을 합리적인 가격에, 승승장구물!
- 일상에 필요한 모든 것을 합리적인 가격에, 승승장구물!

[temperature=1.0]

- 오늘도 승승장구! 당신의 하루를 채우는 스마트한 쇼핑.
- 필요한 건 다 있다, 승승장구물에서 똑똑하게 누리세요!
- 승승장구물 — 매일이 특별해지는 합리적 쇼핑의 시작.

온도 0에서는 답이 거의 고정, 온도 1.0에서는 매번 다릅니다.

4. 다른 생성 파라미터(맛보기)

temperature 외에도 답변을 통제하는 설정이 있습니다. 지금은 이름만 알아두면 됩니다.

- **max_output_tokens**: 답변 최대 길이(토큰). 길이를 강제로 잘라 비용을 통제.
- **top_p / top_k**: temperature 와 비슷하게 "후보 단어 범위"를 좁히는 방법. 보통 temperature 만으로 충분.

```
config = types.GenerateContentConfig(  
    system_instruction="너는 친절한 상담원이다.",  
    temperature=0.3,  
    max_output_tokens=200, # 답변을 200 토큰 이내로 제한  
)
```

- temperature 는 답변의 무작위성을 조절한다. 낮으면 일관, 높으면 다양.
- 분류·도구 선택·계산 = 0, 상담 = 0.3, 창작 = 0.7~1.0.
- max_output_tokens 로 답변 길이(=비용)도 제한할 수 있다.

따라하기 — Gemini 호출의 모든 것

0. 실습 준비 (설치)

```
conda activate agentic

# 실습에 필요한 라이브러리
# - google-genai : Gemini 공식 Python SDK (1 강에서 설치했다면 생략 가능)
pip install google-genai python-dotenv
```

.env 에 GOOGLE_API_KEY 가 설정돼 있어야 합니다. (python code/common.py 로 확인)

1. 공통 시작부

```
# 실행: python code/ch02_llm_api.py
import os, sys, pathlib

# [왜] 실행 위치와 무관하게 같은 폴더(code/)의 common.py 를 import 하려고 경로를 추가.
sys.path.append(str(pathlib.Path(__file__).resolve().parent))

from common import get_genai_client, GEMINI_MODEL

# types = google-genai 의 설정/입력 객체 모음. GenerateContentConfig 등을 여기서 가져온다.
from google.genai import types

client = get_genai_client()
```

2. 재사용할 chat() 함수

시스템 지시와 사용자 입력을 받아 답을 돌려주는 함수입니다.

```
def chat(system: str, user: str, temperature: float = 0.3) -> str:
    """시스템 지시(system)와 사용자 입력(user)을 받아 답변 텍스트를 돌려준다.

    핵심은 config 안의 두 가지다.
```

- system_instruction: 모델의 '역할·말투'를 지정.
- temperature: 생성의 무작위성(0=일관, 높을수록 다양).

```

"""
resp = client.models.generate_content(
    model=GEMINI_MODEL,
    contents=user,
    config=types.GenerateContentConfig(
        system_instruction=system,
        temperature=temperature,
    ),
)
return resp.text

```

3. 기본 호출 + 토큰 확인

```

# 시스템 지시 없이 가장 단순하게 호출. temperature=0.7 로 다양한 홍보 문구를 유도.
resp = client.models.generate_content(
    model=GEMINI_MODEL,
    contents="승승장구물을 한 문장으로 홍보해줘.",
    config=types.GenerateContentConfig(temperature=0.7),
)

print("답변:", resp.text)

# usage_metadata = 이번 호출에 쓴 토큰 수(prompt/응답/합계). 토큰이 곧 과금 기준이다.
print("토큰:", resp.usage_metadata)

```

4. 시스템 지시로 역할 부여

```

# 같은 질문(q)이라도 system_instruction 만 바꾸면 말투·페르소나가 통째로 바뀐다.
q = "환불 얼마나 걸려요?"

print("\n[친절한 상담원]")
print(chat("너는 승승장구물의 친절한 CS 상담원이다. 존댓말로 간결히 답하라.", q))

```

```
print("\n[무뚝뚝한 엔지니어]")  
print(chat("너는 무뚝뚝한 엔지니어다. 핵심만 한 줄로 답하라.", q))
```

5. 전체 실행

```
python code/llm_api.py
```

예상 출력

답변: 일상에 필요한 모든 것을 합리적인 가격에! 승승장구몰에서 만나보세요.
토큰: prompt_token_count=11 candidates_token_count=24 total_token_count=35 ...

[친절한 상담원]

안녕하세요 고객님! 환불은 영업일 기준 평균 3~5 일 정도 소요됩니다 ...

[무뚝뚝한 엔지니어]

환불 3~5 영업일.

6. 무엇을 확인했나

- 한 번의 호출에 **model·contents·config** 를 어떻게 채우는지
- 시스템 지시로 말투가 어떻게 바뀌는지
- **usage_metadata** 로 토큰(=비용)을 어떻게 확인하는지

이 세 가지가 모든 LLM 호출의 기본기입니다.

따라하기 — (보조) OpenAI 호출

목표

같은 "요청-응답" 구조를 **OpenAI(GPT)**로도 호출해 봅니다. 핵심은 "**모델 회사가 달라도 구조는 거의 같다**"를 직접 확인하는 것입니다. 이걸 알면 나중에 모델을 자유롭게 갈아탈 수 있습니다.

0. 실습 준비 (설치)

```
conda activate agentic

# 2 강에서 처음 쓰는 라이브러리
# - openai : OpenAI 공식 Python SDK (GPT 모델 호출용)
pip install openai
```

그리고 .env 에 OPENAI_API_KEY 를 추가합니다.

```
OPENAI_API_KEY=sk-...여러분의_키...
```

1. Gemini 와 OpenAI, 무엇이 같고 다른가

구분	Gemini (google-genai)	OpenAI (openai)
클라이언트	genai.Client(api_key=...)	OpenAI() (키는 환경변수)
호출	client.models.generate_content(...)	client.chat.completions.create(...)
입력 형식	contents="질문" + config 에 시스템 지시	messages=[{role, content}, ...] 리스트
답 꺼내기	resp.text	resp.choices[0].message.content
토큰	resp.usage_metadata	resp.usage

가장 큰 차이는 **입력 형식**입니다. OpenAI 는 시스템 지시와 질문을 **messages** 리스트에 role 로 구분해 함께 담습니다.

2. 코드 — OpenAI 로 같은 질문 던지기

```
import os

# OpenAI 키가 있을 때만 실행 (없으면 건너뛰)
key = os.getenv("OPENAI_API_KEY", "")
```

```

if key and not key.startswith("여기에"):
    from openai import OpenAI
    oai = OpenAI()                                # 키는 환경변수에서 자동으로 읽음

    # OpenAI 는 messages 리스트로 '시스템 지시'와 '질문'을 함께 보낸다.
    r = oai.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "너는 승승장구물 상담원이다."}, # 시스템 지시
            {"role": "user", "content": "환불 얼마나 걸려요?"},          # 사용자 질문
        ],
    )
    print("[OpenAI]", r.choices[0].message.content) # 답 꺼내기
    print("토큰:", r.usage)                         # 토큰 사용량
else:
    print("[OpenAI] OPENAI_API_KEY 미설정 — 생략 (Gemini 만으로 충분합니다)")

```

예상 출력

```

[OpenAI] 안녕하세요! 환불은 영업일 기준 평균 3~5 일 정도 소요됩니다 ...
토큰: CompletionUsage(completion_tokens=28, prompt_tokens=24, total_tokens=52)

```

참고: OpenAI 최신 SDK 는 system 대신 developer role 도 지원합니다. 여기서는 호환성과 직관성을 위해 익숙한 system 을 사용합니다 — 둘 다 정상 동작합니다.

3. messages 리스트, 왜 이렇게 생겼나

messages 는 **대화 기록 전체**를 담는 그릇입니다. 그래서 멀티턴 대화를 만들 때 이렇게 쌓아 갑니다.

```

messages = [
    {"role": "system", "content": "너는 친절한 상담원이다."},
    {"role": "user", "content": "안녕하세요"},
    {"role": "assistant", "content": "안녕하세요! 무엇을 도와드릴까요?"}, # 모델의 이전 답
    {"role": "user", "content": "환불 얼마나 걸려요?"},                  # 이어지는 질문

```

]

이 "리스트에 대화를 쌓는" 방식은 메모리에서 매우 중요해집니다. 지금은 "OpenAI 는 대화를 리스트로 관리한다"만 기억하면 됩니다.

4. 핵심 교훈 — 구조는 같다

회사가 달라도 결국 **요청(모델+입력+설정) → 응답(텍스트+토큰)** 구조는 똑같습니다.

그래서 **LangChain** 이라는 도구를 써서, 이 차이마저 감춰 버립니다. `get_chat(provider="gemini")` 한 줄로 모델을 갈아끼우게 되죠. 그 바탕에는 지금 배운 "구조는 같다"는 원리가 깔려 있습니다.

핵심

- OpenAI 호출은 `client.chat.completions.create(model, messages)` 형태.
- 입력은 **messages 리스트**(role: system/user/assistant)로 담는다.
- 답은 `resp.choices[0].message.content`, 토큰은 `resp.usage`.
- **모델 회사가 달라도 요청-응답 구조는 같다** → 모델 교체가 쉽다.